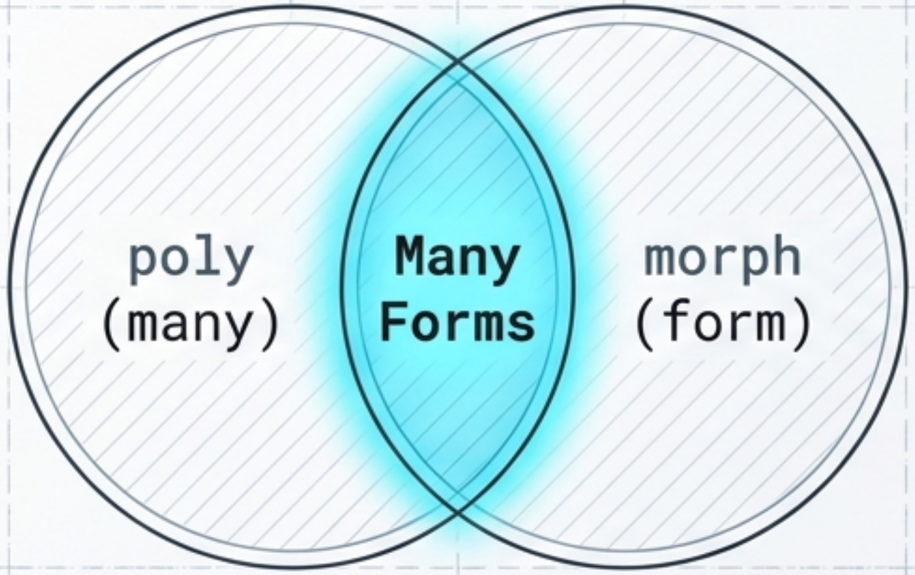
Polymorphism in OOP

BTech-CSE
Unit-3

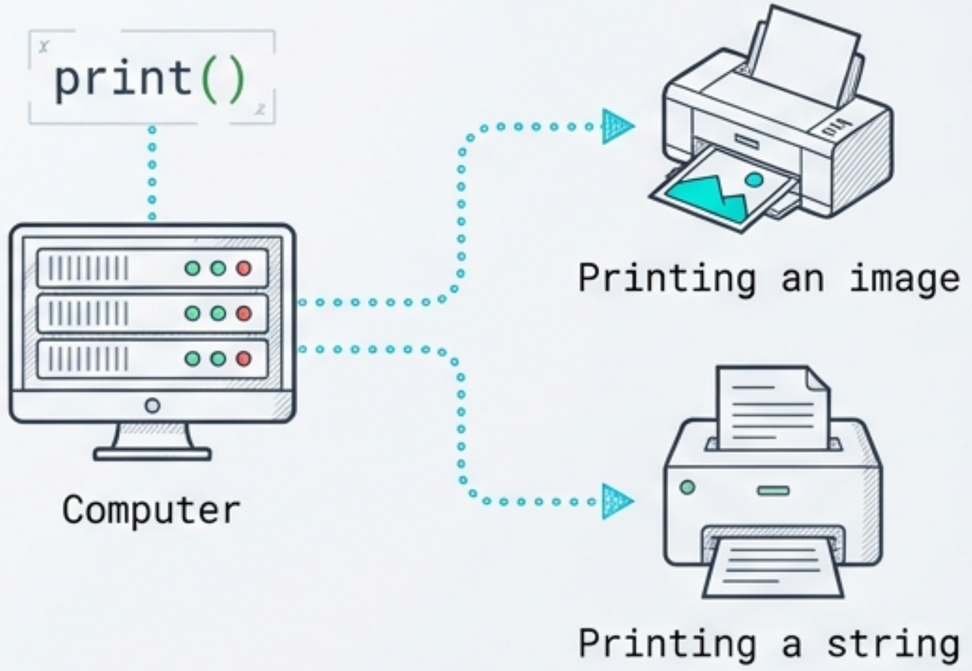
POLYMORPHISM: THE ART OF MANY FORMS

Etymology & Definition



Definition: Allows one function, method, or object to behave differently depending on the situation.

Real-Life Analogy



One command name, different handling based on data type.

Gemini Notebook

Why Polymorphism Matters



Code Flexibility

Replaces rigid designs with extensible, reusable code.



Unified Actions

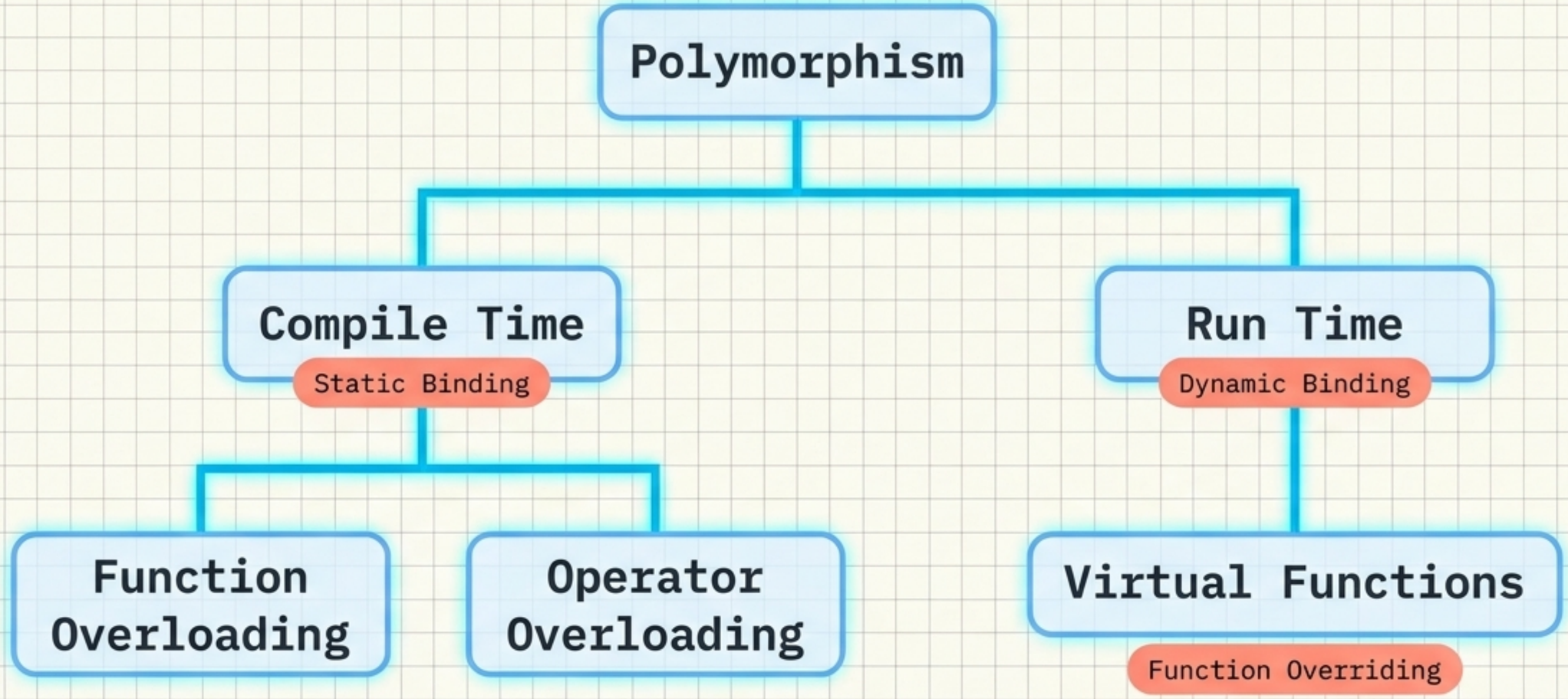
Perform a single action in different ways without needing new command names.



Runtime Adaptability

Programs intelligently decide which specific function to execute at the exact moment of need.

The Polymorphism Landscape



Gemini Notebook

Compile-Time: Function Overloading

Rule 1: Exact same function name.

Rule 2: Different parameters (varying by type or total number).

Rule 3: Occurs entirely within the same scope.

```
1  // 1. Adds two integers
2  int add(int a, int b) {
3    return a + b;
4  }

5  // 2. Adds three integers
6  int add(int a, int b, int c) {
7    return a + b + c;
8  }

9  // 3. Adds two doubles
9  double add(double a, double b) {
10   return a + b;
11 }
```

Gemini Notebook

Compile-Time: Operator Overloading

Redefining standard **mathematical** operators (+, -, *).

Allows standard operators to **work seamlessly with custom objects**.

Makes object math **readable, natural, and logical**.

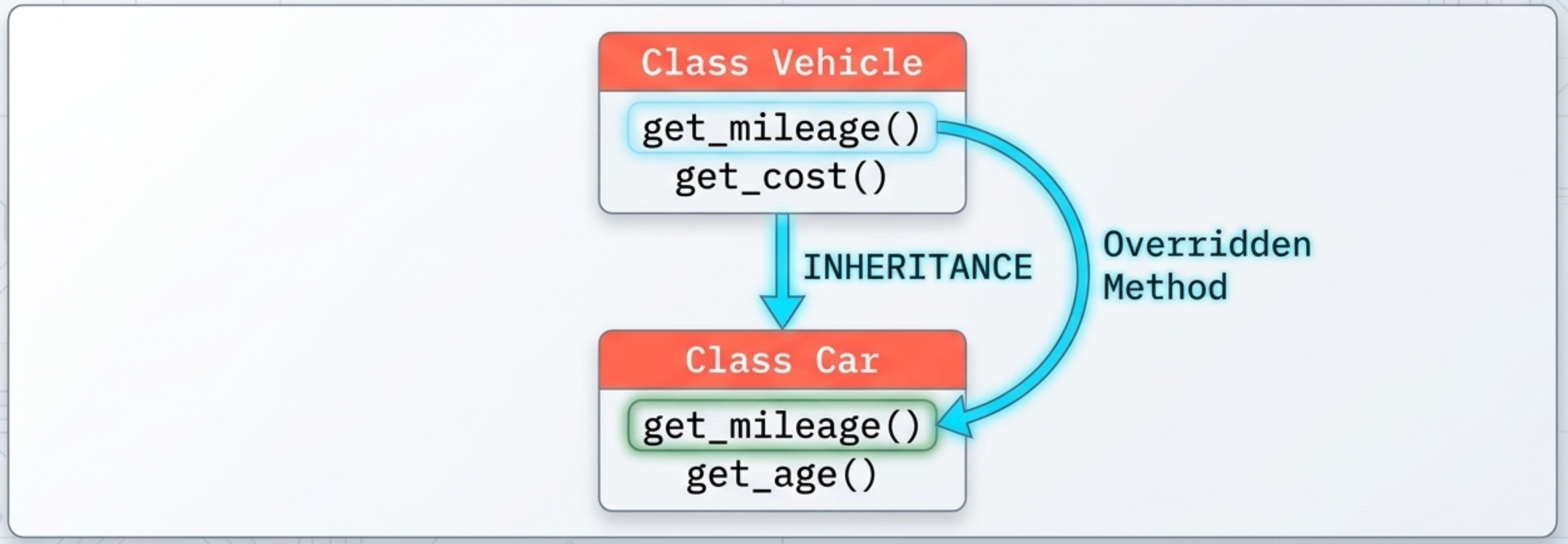
```
class Complex {
    int real, imag;
public:
    Complex(int r=0, int i=0) { real=r; imag=i; }

    // Overloading the '+' operator
    Complex operator + (Complex obj) {
        Complex res;
        res.real = real + obj.real;
        res.imag = imag + obj.imag;
        return res;
    }
};
```

Gemini Notebook

Run-Time: Function Overriding

Redefining a base class function inside a derived class.
Must have the exact same name, parameters, and return type.



Gemini Notebook

The Catalyst: Virtual Functions

The Issue

Pointers normally execute functions based on the pointer's data type, ignoring the actual object assigned to it.

```
virtual void sound();
```

The Result

Forces the compiler to wait until runtime to execute the correct function based on the actual object in memory (Dynamic Binding).

The Fix

Declare the base class function with the "virtual" keyword.

Gemini Notebook

Run-Time Polymorphism in Action

```
class Animal {
public:
    virtual void sound() { cout << "Animal" << endl; }
};

class Dog : public Animal {
public:
    void sound() override { cout << "Dog barks" << endl; }
};

int main() {
    Animal a;
    Dog d;
    Animal* ptr;
    ptr = &a;
    ptr->sound();
    ptr = &d;
    ptr->sound();
}
```

Pointer looks at the 'virtual' tag, sees an Animal object, and calls Animal's sound().

Pointer dynamically binds to the Dog object in memory, triggering the overridden sound().

Gemini Notebook

Synthesis: Overloading vs. Overriding

Feature	Function Overloading	Function Overriding
Parameters	Different parameters	Exact same parameters
Occurs in	Same class	Different classes (Inheritance)
Binding	Compile-time (Static)	Run-time (Dynamic)
Keyword	None	`virtual` (in C++)

Gemini Notebook